

MODULE 4 LAB WORKBOOK

Robot Framework – Test Automation

Name :Debasmita Majhi

Module: Module 4:> Robot Framework

Module 4 – Lab Work Record

This workbook documents the eight Module 4 Robot Framework experiments. Each experiment contains the problem statement, objective, tools/software/concepts, source code, implementation steps, output placeholder, result, observation, and conclusion.

Experiment 4.1 – Basic Syntax & Keywords

1. Problem Statement

Create and execute a basic Robot Framework web automation test using standard keywords.

2. Objective

Understand Robot Framework syntax and practice basic browser automation and verification.

3. Tools, Software, and Concepts Used

Robot Framework, Python, VS Code, SeleniumLibrary, Google Chrome, keywords, element verification.

4. Source Code

```
*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://automationexercise.com/products
${BROWSER}   Chrome

*** Test Cases ***
Basic Robot Framework Web Test
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Wait Until Element Is Visible    xpath=//input[@id='search_product']    20s
    Page Should Contain Element    xpath=//input[@id='search_product']
    Input Text    xpath=//input[@id='search_product']    Blue Top
    Click Button    xpath=//button[@id='submit_search']
    Wait Until Element Is Visible    xpath=//h2[contains(translate(., 'abcdefghijklmnopqrstuvwxyz', 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'), 'SEARCHED PRODUCTS')]    20s
    Log    Product search completed successfully
    Close Browser
```

5. Implementation Steps

1. Create a .robot file.
2. Import SeleniumLibrary.
3. Define URL and browser variables.
4. Open the products page and verify the search field.
5. Search for Blue Top and verify the searched-products heading.
6. Close the browser.

6. Output

Screenshot file: Experiment_4_1_Result.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.1 – Execution Output

7. Result

Test executed successfully.

8. Observation

Robot Framework keywords provide readable web-automation steps.

9. Conclusion

Basic syntax, SeleniumLibrary keywords, and verification were successfully practiced.

Experiment 4.2 – Variables & Data-Driven Automation

1. Problem Statement

Use variables and multiple sets of input data to perform repeated login-page automation.

2. Objective

Understand Robot Framework variables and simple data-driven execution.

3. Tools, Software, and Concepts Used

Robot Framework, Python, VS Code, SeleniumLibrary, Chrome, variables, FOR loop, IN ZIP.

4. Source Code

```
*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://automationexercise.com/login
${BROWSER}   Chrome
${SCREENSHOT_DIR}  ${CURDIR}${/}screenshots

@{EMAILS}    test1@example.com    test2@example.com    test3@example.com
@{PASSWORDS} password1    password2    password3

*** Test Cases ***
Data Driven Login Test
    FOR    ${email}    ${password}    IN ZIP    ${EMAILS}    ${PASSWORDS}
        Open Browser    ${URL}    ${BROWSER}
        Maximize Browser Window
        Wait Until Element Is Visible    xpath=//input[@data-qa='login-email']    15s
        Input Text    xpath=//input[@data-qa='login-email']    ${email}
        Input Text    xpath=//input[@data-qa='login-password']    ${password}
        Log    Testing login with ${email}
        Capture Page Screenshot    ${SCREENSHOT_DIR}${/}Experiment_4_2_${email}.png
        Close Browser
    END
```

5. Implementation Steps

7. Define email and password list variables.
8. Use FOR with IN ZIP to pair the values.
9. Open the login page for each data set.
10. Enter the data and capture a screenshot.
11. Close the browser after each iteration.

6. Output

Screenshot file: Experiment_4_2_<email>.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.2 – Execution Output

7. Result

Data-driven test executed successfully and screenshots were captured.

8. Observation

List variables and IN ZIP allow multiple data sets to use the same test logic.

9. Conclusion

Variables and data-driven automation were successfully implemented.

Experiment 4.3 – Custom Keywords & Libraries

1. Problem Statement

Create and use a custom Python keyword from Robot Framework and combine it with standard libraries.

2. Objective

Understand Python keyword integration and standard Robot Framework libraries.

3. Tools, Software, and Concepts Used

Robot Framework, Python, BuiltIn, String library, custom Python keyword, library integration.

4. Source Code

```
*** Settings ***
Library      BuiltIn
Library      String
Library      ${CURDIR}${/}python_keywords${/}custom_keywords.py

*** Test Cases ***
Custom Python Keyword Test
    ${result}=    Calculate Sum      10      20
    Should Be Equal As Integers     ${result}    30
    Log      Sum calculated using Python custom keyword: ${result}

BuiltIn String And Math Test
    ${text}=    Set Variable      robot framework
    ${upper_text}=    Convert To Upper Case    ${text}
    Should Be Equal    ${upper_text}    ROBOT FRAMEWORK
    ${number}=    Evaluate    10 + 20
    Should Be Equal As Integers    ${number}    30
    Log      String and mathematical operations completed successfully
```

Supporting Source Code

Supporting file: python_keywords/custom_keywords.py

```
def calculate_sum(first_number, second_number):
    return int(first_number) + int(second_number)
```

5. Implementation Steps

12. Create the Python custom keyword file.
13. Import the Python file.
14. Call Calculate Sum with 10 and 20.
15. Verify the result is 30.
16. Use String and BuiltIn operations.
17. Run the test.

6. Output

Screenshot file: Experiment_4_3_Result.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.3 – Execution Output

7. Result

Both tests passed successfully.

8. Observation

Robot Framework can expose Python functions as readable keywords.

9. Conclusion

Custom Python keywords and libraries were successfully integrated.

Experiment 4.4 – Assertions & Verification

1. Problem Statement

Verify expected conditions in web UI automation and an API response.

2. Objective

Understand UI and API assertions for validating expected results.

3. Tools, Software, and Concepts Used

Robot Framework, SeleniumLibrary, RequestsLibrary, Python, Chrome, UI assertions, API assertions.

4. Source Code

```
*** Settings ***
Library      SeleniumLibrary
Library      RequestsLibrary

*** Variables ***
${URL}       https://automationexercise.com/products
${BROWSER}   Chrome
${SCREENSHOT_DIR}  ${CURDIR}${/}screenshots

*** Test Cases ***
UI Assertion Test
    Open Browser      ${URL}      ${BROWSER}
    Maximize Browser Window
    Wait Until Element Is Visible    xpath=//input[@id='search_product']    20s
    Page Should Contain Element    xpath=//input[@id='search_product']
    Input Text    xpath=//input[@id='search_product']    Blue Top
    Click Button    xpath=//button[@id='submit_search']
    Wait Until Element Is Visible    xpath=//div[@class='productinfo text-center']    20s
    Page Should Contain Element    xpath=//div[@class='productinfo text-center']
    Log    Product search assertion passed successfully
    Capture Page Screenshot    ${SCREENSHOT_DIR}${/}Experiment_4_4_UI.png
    Close Browser

API Response Assertion Test
    Create Session    jsonplaceholder    https://jsonplaceholder.typicode.com
    ${response}=    GET On Session    jsonplaceholder    /posts/1
    Should Be Equal As Strings    ${response.status_code}    200
    Log    API response status verified successfully
```

5. Implementation Steps

18. Open the products page.
19. Verify the search field.
20. Search for Blue Top and verify product information.
21. Create an API session.
22. Send a GET request to /posts/1.
23. Verify status code 200.

6. Output

Screenshot file: Experiment_4_4_UI.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.4 – Execution Output

7. Result

UI and API assertion tests passed successfully.

8. Observation

Assertions verify that actual results meet expected conditions.

9. Conclusion

UI and API verification were successfully demonstrated.

Experiment 4.5 – Setup & Teardown

1. Problem Statement

Use Test Setup and Test Teardown to prepare and clean up the browser around a test.

2. Objective

Understand reusable setup and teardown operations.

3. Tools, Software, and Concepts Used

Robot Framework, SeleniumLibrary, Python, Chrome, Test Setup, Test Teardown, reusable keywords.

4. Source Code

```
*** Settings ***
Library      SeleniumLibrary

Test Setup    Open Application
Test Teardown Close Application

*** Variables ***
${URL}        https://automationexercise.com/login
${BROWSER}    Chrome

*** Keywords ***
Open Application
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window

Close Application
    Close All Browsers

*** Test Cases ***
Verify Login Page
    Wait Until Element Is Visible    xpath=//input[@data-qa='login-email']    20s
    Page Should Contain Element    xpath=//input[@data-qa='login-email']
    Page Should Contain Element    xpath=//input[@data-qa='login-password']
    Log    Login page verified successfully
    Capture Page Screenshot    ${CURDIR}${/}screenshots${/}Experiment_4_5_Result.png
```

5. Implementation Steps

24. Create Open Application and Close Application keywords.
25. Set Open Application as Test Setup.
26. Set Close Application as Test Teardown.
27. Verify login fields.
28. Capture the screenshot.
29. Execute the test.

6. Output

Screenshot file: Experiment_4_5_Result.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.5 – Execution Output

7. Result

Login-page verification passed with setup and teardown.

8. Observation

Setup and teardown reduce repeated browser-management steps.

9. Conclusion

Test Setup and Test Teardown were successfully implemented.

Experiment 4.6 – Tags & Test Execution

1. Problem Statement

Organize test cases using tags and execute selected groups of tests.

2. Objective

Understand tags and selective execution using include and exclude options.

3. Tools, Software, and Concepts Used

Robot Framework, SeleniumLibrary, Python, Chrome, tags, smoke, regression.

4. Source Code

```
*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://automationexercise.com
${BROWSER}   Chrome

*** Test Cases ***
Verify Home Page
    [Tags]    smoke
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Wait Until Page Contains    Automation Exercise    20s
    Log    Home page verified successfully
    Capture Page Screenshot    ${CURDIR}${/}screenshots${/}Experiment_4_6_Home.png
    Close Browser

Verify Products Page
    [Tags]    regression
    Open Browser    ${URL}/products    ${BROWSER}
    Maximize Browser Window
    Wait Until Element Is Visible    xpath=//input[@id='search_product']    20s
    Page Should Contain Element    xpath=//input[@id='search_product']
    Log    Products page verified successfully
    Capture Page Screenshot    ${CURDIR}${/}screenshots${/}Experiment_4_6_Products.png
    Close Browser
```

5. Implementation Steps

30. Assign smoke to the home-page test.
31. Assign regression to the products-page test.
32. Run all tests.
33. Run only smoke using robot --include smoke experiment_4_6.robot.
34. Run tests excluding smoke using robot --exclude smoke experiment_4_6.robot.
35. Review results and screenshots.

6. Output

Screenshot file: Experiment_4_6_Home.png and Experiment_4_6_Products.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.6 – Execution Output

7. Result

Tagged tests executed successfully, including selective execution.

8. Observation

Tags organize tests and allow selective execution.

9. Conclusion

Tags and include/exclude execution were successfully demonstrated.

Experiment 4.7 – Reports & Logs

1. Problem Statement

Generate execution logs, reports, and a screenshot for a Robot Framework web test.

2. Objective

Understand Robot Framework reporting and logging.

3. Tools, Software, and Concepts Used

Robot Framework, SeleniumLibrary, Python, Chrome, HTML reports, logs, screenshots.

4. Source Code

```
*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://automationexercise.com
${BROWSER}   Chrome

*** Test Cases ***
Generate Report And Log
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window
    Log    Browser opened successfully
    Wait Until Page Contains    Automation Exercise    20s
    Log    Automation Exercise home page verified successfully
    Capture Page Screenshot    ${CURDIR}${/}screenshots${/}Experiment_4_7_Result.png
    Log    Screenshot captured successfully
    Close Browser
    Log    Test execution completed successfully
```

5. Implementation Steps

36. Open Automation Exercise.
37. Log execution messages.
38. Verify the home page.
39. Capture a screenshot.
40. Close the browser.
41. Inspect output.xml, log.html, and report.html.

6. Output

Screenshot file: Experiment_4_7_Result.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.7 – Execution Output

7. Result

Test passed and output.xml, log.html, and report.html were generated.

8. Observation

Robot Framework reports and logs help review execution results.

9. Conclusion

Reporting and logging were successfully demonstrated.

Experiment 4.8 – Resources & Page Objects / Readability

1. Problem Statement

Use a separate resource file containing reusable keywords to improve readability and maintainability.

2. Objective

Understand resource files and reusable keywords.

3. Tools, Software, and Concepts Used

Robot Framework, SeleniumLibrary, Python, Chrome, resource files, reusable keywords, setup/teardown.

4. Source Code

```
*** Settings ***
Resource      resources.robot

Test Setup    Open Application
Test Teardown Close Application

*** Test Cases ***
Verify Home Page Using Resource Keywords
    Verify Home Page
    Capture Page Screenshot    ${CURDIR}${/}screenshots${/}Experiment_4_8_Result.png
```

Supporting Source Code

```
Supporting file: resources.robot

*** Settings ***
Library      SeleniumLibrary

*** Variables ***
${URL}       https://automationexercise.com
${BROWSER}   Chrome

*** Keywords ***
Open Application
    Open Browser    ${URL}    ${BROWSER}
    Maximize Browser Window

Close Application
    Close All Browsers

Verify Home Page
    Wait Until Page Contains    Automation Exercise    20s
    Log    Home page verified successfully
```

5. Implementation Steps

42. Create resources.robot with reusable browser and verification keywords.
43. Import resources.robot.
44. Use Open Application as Test Setup.
45. Use Close Application as Test Teardown.
46. Call Verify Home Page.
47. Capture the screenshot and execute the test.

6. Output

Screenshot file: Experiment_4_8_Result.png

[PASTE YOUR ACTUAL EXECUTION SCREENSHOT HERE]

Figure: Experiment 4.8 – Execution Output

7. Result

1 test passed and 0 failed.

8. Observation

Resource files reduce duplication and keep test cases concise and readable.

9. Conclusion

Reusable resource keywords were successfully implemented.

Overall Conclusion

The Module 4 experiments provided practical experience with Robot Framework for web and API test automation. The work covered basic syntax and keywords, variables and data-driven execution, custom Python keywords and libraries, assertions and verification, setup and teardown, tags and selective execution, reports and logs, and reusable resource files.

Screenshot Checklist

- Experiment_4_1_Result.png – actual screenshot inserted and clearly labelled.
- Experiment_4_2_<email>.png – actual screenshot inserted and clearly labelled.
- Experiment_4_3_Result.png – actual screenshot inserted and clearly labelled.
- Experiment_4_4_UI.png – actual screenshot inserted and clearly labelled.
- Experiment_4_5_Result.png – actual screenshot inserted and clearly labelled.
- Experiment_4_6_Home.png – actual screenshot inserted and clearly labelled.
- Experiment_4_6_Products.png – actual screenshot inserted and clearly labelled.
- Experiment_4_7_Result.png – actual screenshot inserted and clearly labelled.
- Experiment_4_8_Result.png – actual screenshot inserted and clearly labelled.